Estructura de Datos y Algoritmos

Jaime Nazar Anchorena

Contents

Unidad 1: Algoritmos **4**
- Tiempo de ejecucion . . . 4
  - Empiricamente . . . 4
  - Teoricamente . . . 4
  - Calculo del Big O . . . 4
- Espacio se RAM . . . 5
  - Empiricamente . . . 5
  - Teoricamente . . . 6
- Maven . . . 6
  - Goals . . . 6
  - Build Phases . . . 6
  - `pom.xml` . . . 6
  - Como correr el `.jar` fuera del entorno . . . 7
- Test-Driven Development . . . 7
  - Unit Testing . . . 7
  - Como testear en un Unit Test . . . 8
  - JUnit 5 . . . 8
  - Comparando resultados . . . 8
  - Ambiente de prueba . . . 9
  - JUnit Coverage . . . 10

Unidad 2: Algoritmos para textos **10**
- Definicion: Alfabeto . . . 10
- Definicion: String . . . 10
- Por que se lo denota λ? . . . 10
- Definicion: Conjunto de todos los string posibles . . . 10
- Definicion: Concatenacion de strings . . . 10
- Definicion: Prefijos, Sufijos y Subtrings . . . 10
- Definicion: Bordes . . . 11
- Dara Quality - Matching . . . 11
  - Minimas reglas que deberian aplicarse: . . . 11
  - Reglas especificas para un determinado tipo de datos . . . 11
- Algoritmos busqueda aproximada . . . 11
  - Soundex . . . 11

Metaphone . . . 13
Levenshtein Distance . . . 13
Optimizacion - Version compacta . . . 15
Q-Grams(o N-Grams) . . . 15
Algoritmos para busqueda exacta . . . 16
Fuerza bruta o Naive . . . 16
Knuth-Morris-Pratt . . . 16
Fulltext Retrieval . . . 17
Indices . . . 18
Definicion: Documento . . . 18
Definicion: Campo(Field) . . . 18
Definicion: Termino(Term) . . . 18
Almacenamiento . . . 19
Ranking de documentos . . . 20
Queries . . . 20
Analyzers . . . 21
Solr y Elasticsearch . . . 21

Unidad 3: Estructuras lineales **22**
Arreglos . . . 22
Indices y estructuras posibles . . . 22
Teorema maestro . . . 22
Expansion recursiva . . . 23
Complejidad para version iterativa y para recursiva . . . 23
Metodos de ordenamiento . . . 23
Generics en Java . . . 24
Como incluir `.jar` . . . 25
Stack . . . 25
Implementacion . . . 25
Caso de uso: Evaluador de expresiones . . . 25
Infija a posfija . . . 26
Cola . . . 27
Definicion: Queue . . . 27
Lista . . . 27
Lista lineal simplemente encadenada . . . 27
Insertar . . . 28
Iterador . . . 28

Unidad 4: Hashing **28**
Hash table . . . 28
Primer escenario . . . 29
Definicion: Funcion hash perfecta . . . 29
Definicion: Colision . . . 29
Factor de carga(current load factor) . . . 29
Hashing sobre strings . . . 29
Funcinoes para claves numericas . . . 30

Manejo de colisiones . . . 30
Open Addressing . . . 30

Unidad 5: Arboles binarios **31**
Arboles de expresiones . . . 31
Caracteristicas . . . 31
Aclaracion . . . 31
Formalmente . . . 31
Arboles binarios en general . . . 32
Algoritmo . . . 32
Definicion: Arbol binario completo(complete) . . . 33
Definicion: Altura . . . 33
Arboles de busqueda - BST . . . 33
Operaciones . . . 33
Arboles balanceados . . . 34
Algoritmo AVL . . . 34
Red-Black Tree . . . 35
Algunas definiciones . . . 36
Arbol B de orden N . . . 37

Unidad 6: Grafos **38**
Tipos . . . 38
No dirigidos . . . 38
Dirigidos(digrafos) . . . 39
Con pesos . . . 39
Implementacion . . . 39
Testeos de unidad . . . 40
Recorridos . . . 40
Breadth-First Search (BFS) . . . 40
Depth-First Search(DFS) . . . 40
Dijkstra . . . 40

Unidad 1: Algoritmos

El analizis de algoritmos nos permite caracterizar la cantidad de recursos computacionales el usara un algoritmo dado, estas son:

1. Tiempo de ejecucion
2. Espacio que utilizan

En caso de considerar si un algoritmo es mejor que otro utilizando tanto la complejidad temporal como la espacial, pero generalmente ocurre que un algorimo es mejor espacialmente y peor temporalmente y viceversa. Hay que considerar cuales son los **tradeoff** y decidir en base a mi situacion.

> **Nota**: Una vez arman un algoritmo, no lo dejen asi, busquen maneras de mejorarlo.

Tiempo de ejecucion

Como los mido? Hay dos caracteristicas, ambas sirven, pero apuntan a cosas diferentes

1. Empiricamente
2. Teoricamente

Empiricamente

Debe ocurrir en la misma maquina, se puede haver con las clases `Timer` que hicimos en clase.

> **NOTA:** Cuidado, a veces un algorimo parece mejor que otro, pero en casos extremos puede ser peor o incluso fallar.

Teoricamente

Da una tendencia, no un tiempo dado. Categoriza los algoritmos, por ejemplo, todos los O(n^2). Consiste en una descripcion de alto nivel del algoritmo para evaluar su eficiencia, idependientemente del hardware y el software donde se ejecuto.

Para contar las operaciones y encontrar Big O, hay que contar las operaciones fijas y los ciclos. Hay que tener en cuenta el peor de los casos(siempre, por lo menos en esta materia) y encontrar la menor O grande. Si aparece una llamada a una funcion, hay que revisar la documentacion para verificar el numero de operaciones que requiere.

Calculo del Big O

La descripcion que buscamos para comparar algoritmos es una asintota(cota). Comportamiento asintotico superior o Big O se define como:

Sean T(N) y g(N) funciones con N > 0

Se dice que T(n) es O(g(N)) si Existe c > 0(no depende de N) y Existe n_c > 0 tal que Para Todo N >= n se cumple que 0 <= T(N) <= c * g(N). Ejemplo completo del campus:

Basicamente, lo importante es encontrar el T(N) y de ahi se puede deducir O grande sin formalizar. Para calcular la complejidad temporal, no basta con analizar la cantidad de datos de entrada, la performance tambien puede depender de como vienen los datos.

Definición de Comportamiento asintótico superior u O grande (*asymptotic upper bound running time u O-notation*) de un algoritmo.

Sean T(N) y g(N) funciones con N >0.
Se dice que T(n) es O (g(N)) sii $\exists$ c > 0 (constante no dependiente de N) y $\exists$ n_0 > 0 tal que $\forall$ N $\geq$ n_0 se cumple que 0 $\leq$ T(N) $\leq$ c * g(N).

Ej: En nuestro caso, algoA tiene T(N) = 3 * N
Pero para hacer más complicada la discusión, supongamos que T(N) = 1 + 3 * N, o sea g(N) puede ser N.
O sea, si 0$\leq$ 1+3*N $\leq$ c * N ¿cuánto debe valer c?

0$\leq$ 1/N + 3 $\leq$ c
Si N es 2 entonces c >= 3.5
Si N es 5 entonces c >= 3.2, mejor la cota.
Si N es 500, c >= 3.002, mejor aún.
Yo quiero N$\rightarrow\infty$, entonces c >= 3. Así, el orden de algoA es O(N).

Encontré caracterizar una constante c, que es 3, para el cual ser verifica la fórmula para N$\rightarrow\infty$ ($\forall$ N $\geq$ n_0). El algoritmo es O(N).

Figure 1: Calculo formal del O grande

Espacio se RAM

Similar al tiempo, existen metodos empiricos y teoricos para medir el espacio que un algoritmo ocupa en memoria, la complejidad espacial. Un programa presenta, datos, direcciones de retorno, alocaciones en memoria, etc.

Empiricamente

Para caracterizarlo en Java, hay que tener en cuenta el spacio que se aloca en:

- **Heap**: Cada vez que hacemos un `new` reservamos memoria en esta zona. Luego el GC libera esta zona cuando detecta que ya no esta siendo referenciada por alguna variable.
- **Stack**: Cada vez que se invoca un metodo se genera un **stack frame** para el mismo, conteniendo los parametros y variables temporales, asi como el lugar de la proxima instruccion a ejecutar(para que continue la ejecucion una vez termine la funcion).

Java permite configurar al heap con parametros, indicando la cantidad inicial de heap prelocada y la cantidad maxima posible de aloca. Al momento de necesitar mas recursos, se indica con los siguientes valores, dependen del equipo(cantidad de memoria, etc):

```
cd target
```

Correr el programa como:

```
java -Xms512m -Xmx4G -cp HeapOverflow-1.jar space.Generate
```

Para que el IDE lo haga automaticamente, agregar en VM args. En mi computadora:

Parametros	Heap Overflow en n
-Xms512m -Xmx4G	2043
-Xms512m -Xmx16G	9200+

Por otro lado, observando un programa recursivo en el stack:

Parametros	Stak Overflow en n
-Xss10k	Stack too small
-Xss1G	49 and I got bored

Teoricamente

Se miden en unidades, un puntero es una unidad, una variable es una unidad. No depende de la cantidad de variables su estas son siempre las mismas, si un algoritmo presenta una complejidad espaacial de 3 unidades, entonces es de orden O(1) (no seria el caso de un algorimo que usa recursividad).

Maven

Maven es un manejador de proyectos en Java, como objetivos propone:

- Proporcionar un sistema de construccion uniforme
- Proporcionar informacion de calidad del proyecto
- Proporcionar pautas para el desarrollo de mejores precticas.
- Permitir la migracion transparente a nuevas funcionalidades.

Como su funcionalidad principal, permite declarar **dependencias** para utilizar librerias externas o nuestras. Maven se encarga de compilar el proyecto, armar el `.jar` con lo que ya compile, y corre los test. Tambien, guarda los projectos localmente para poder utilizar dentro de otros proyectos.

Maven funciona de manera algoritmica, primero busca localmente la dependencia en nuestro repositorio local. Caso contrario, lo busca en internet, si lo encuentra, lo descarga.

Goals

Tareas especificas dentro del build, por ejemplo:

- `mvn jar.jar`: Arma un `/jar` desde codigo ya compilado
- `mvn dependency:tree`: Muestra las dependencias
- `mvn exec java`: Corre el proyecto, `exec` es el plugin y `java` es el goal.

Build Phases

Etapas del armado del proyecto, por ejemplo: - `mvn compile` - `mvn test`

`pom.xml`

Aca declaro como sera my libreria, como aparecera para los demas en el repositorio. Ademas, se le pueden agregar dependencias que tu codigo utilize.

```
    <groupId>ar.edu.itba.eda</groupId>
    <artifactId>TimerJoda</artifactId>
    <version>2</version>
```

Donde `ar.edu.itba.eda` no es el nombre del paquete donde se encuentra el codigo, sino que es el directorio donde se guarda la libreria en el repositorio.

Como correr el `.jar` fuera del entorno

Fuera del entorno de desarrollo, tenemos que poder correr nuestro programa(como el cliente). Para poder compilar todo el entorno en el archivo, hay que agregar un plugin en el `pom.xml`. Lo que va a hacer es juntar todas las dependencias en un unico `.jar`, es basicamente instrucciones extra. Logicamente, el ejecutable generado ocupara mucho mas espacio.

Se puede ejecutar especificando donde esta la clase main.

```
java -cp ***.jar core.AnalisysTimer
```

Pero tambien es posible simplemente decirle que es un ejecutable, pero para que sepa donde esta el main hay que ponerlo en el manifest(independiente de maven), sino da error. Ademas, maven tiene la opcion de agregar en la seccion de plugin para que haga eso automaticamente.

```
<configuration>

    ...

    <archive>
        <manifest>
            <mainClass>core.AnalisysTimer</mainClass>
        </manifest>
    </archive>

    ...

</configuration>
```

Donde core es el package y AnalisysTimer es la clase. Luego, puedo correr my programa con:

```
java -jar ***.jar
```

Test-Driven Development

- TDD es una metodologia que comienza por los test y luego pasa a la implementacion.
- Permite enfocarse en la definicion de la interfaz y del comportabmiento y no en los detalles de implementacion.
- Ve al sistema que se va a desarrollar como una caja negra que todavia no existe.
- Apunta a que todas las funcionalidades tengan algun test.

Unit Testing

Para testear pertes individuales del programa.

- Automatico
- Verifican un unico caso por test
- Repetible
- Independientes de otros test o de condiciones externas.

Como testear en un Unit Test

- Caso de metodo o funcion:
  - Casos tipicos
  - Casos de borde
  - Casos de error
  - Casos de excepcion
- Caso de una clase:
  - Secuencias de llamadas validas
  - Secuencias de llamadas invalidas
  - Chequeo de invariantes

JUnit 5

Framework de java para realizar pruebas de desarrollo de una aplicacion, ha muchos, pero vamos a usar este. Puede comparar resultados de las invocaciones de metodos, o verificar si una excepcion fue lanzada o no. Ademas, un caso de prueba es abortado ni bien falla alguna verificacion.

Para que funcione con maven y poder hacer `mvn test`, hay que agregar un plugin:

```
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-surefire-plugin</artifactId>
    <version>3.5.2</version>
</plugin>
```

Y en dependencias:

```
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-engine</artifactId>
    <version>5.12.0</version>
    <scope>test</scope>
</dependency>
```

Comparando resultados

El test unitario mas simple consiste en comparar el resultado obtenido con el resultado esperado, para ello se puedde utilizar metodos estaticos como `asserEquals(valorEsperado, valor Obtenido)` o `assertTrue()`.

Ademas, si un metodo lanza una excepcion, el mismo se considera que fallo. Si se espera que lanze una excepcion, si indica como `assertThrows(RuntimeException.class, () -> ...)`. Tambien puede teatear y comparar valores de punto flotante, basicamente tiene funciones para casi todos los casos de pruebas.

Para Maven, en las clases con los test, hay que agregar la palabra clave `Test` en el nombre de la clase, de

esa manera Maven va a carrer automaticamente los test en el deployment. Tambien, es obligatorio agregar un `@test`.

```
class TimerTest {

    @test
    @DisplayName("Probar el timer") <- opcional
    void getDurationExceptionTest() {
        ....

        assert...();
    }
}
```

NOTA: Testear Strings es muy dificil(no recomendado), fallan por la mas minima diferencia.

Ambiente de prueba

Frecuentemente se desea tener un ambiente de prueba perfijado(no tener que declarar la misma variable todo el tiempo), para eso se usan anotaciones especiales:

- `@BeforeAll` - Se ejecuta entes de todos los casos de prueba
- `@BeforeEach` - Se ejecuta antes de cada `@Test`
- `@AfterEach`
- `@AfterAll`
- y mas.

Ejemplo:

```
@BeforeAll
static void initAll() {
    ...
}

@AfterEach
void tearDown() {
    ...
}
```

Teniendo en cuenta `MyTimer`, podriamos hacer:

```
private MyTimer timer;

@BeforeEach
void instanceTimer() {
    this.timer = new MyTimer();
}
```

JUnit Coverage

Como verifico que el testeo recorra todo el codigo?

JUnit contempla eso, si quiero ver la covertura de mis test, JUnit generara un reporte.

Unidad 2: Algoritmos para textos

La pricipal motivacion de estos algoritmos es analizar y comparar textos, ya sea para detectar copias o para detectar similitudes. Esto se aplica en areas como en la politica y la biologia. Otra razon mas simple, es para buscar typos y corregirlos. La matematica es muy util para formalizar todo esto.

Definicion: Alfabeto

Se denota Σ y es el conjunto de simbolos o caracteres.

Definicion: String

Dado un alfabeto Σ y $k_{\geq 0} \in N$, un string `S` es un elemento de Σ^k

La longitud del string es k y si $k = 0$ entonces se lo denota λ.

Por que se lo denota λ?

Oara evitar los problemas que tienen los compiladores, un meta-simbolo no deberia ser al mimso tiempo parte del alfabeta. Los lenguages de programacion violan estas relgas y ahi surgen los problemas.

Para evitar ambiguedades obliga a escapar al simbolo comillas dobles cuando participa del string, es decir, `"hola"que"` para el compilador es simplemente 'hola', pero `"hola\"que"` es 'hola"que'. Sin embargo, esto no soluciona la ambiguedad pues '\' es parte del alfabeto. Si no nos presenta confucion, podemos hablar de `""` como el string vacio.

Definicion: Conjunto de todos los string posibles

Dado un alfabeto Σ, $\Sigma^{@} = \cup\Sigma^k$ con $k_{\geq 0} \in N$

> **Nota**: Deberia ser sigma^asterisco, lo reemplace con @

Definicion: Concatenacion de strings

Dado un alfabeto Σ, y $u, w \in \Sigma^{@}$

Se llama **concatenacion** al string definido como uw (uno a continuacion del otro).

Definicion: Prefijos, Sufijos y Subtrings

Dado un alfabeto Σ y los strings $x, w, z \in \Sigma^{@}$. Sea $p = xyz$. Se dice que x es un **prefijo** de p. Se dice que w es un **substring** de p. Se dice que x es un **sufijo** de p.

> λ representa vacio, entonces es sufijo, prefijo y borde.

Definicion: Bordes

Dado un alfabeto Σ y los string $x, y, z \in \Sigma^{@}$. Si $p = wz = zw$ donde $|x| = |z|$, se dice que w es un **borde** de p.

> Un borde es prefijo y sufijo al mismo tiempo.

Dara Quality - Matching

Busqueda aproximada, los buscadores pueden usar varias tecnicas para esto y suelen ser muy variadas. Algunas de ellas(las estrategias pueden ser combinaciones de uno o mas de estas), de menos a mas avanzadas:

- Busca palabras y si las palabras no estan en el corpus de los documentos indizados, encontrar las que mayor similitud posean y sugerirlas.
- Tomar el idioma que tiene configurado el browser para saber en que corpus buscar las palabras usadas. Hay reglas conocidas por el idioma en cuestion, lo cual permite su deteccion.
- Sabiendo que los usuarios buscan palabras y cuando fue un error de tipeo, no clickean nada del resultado e intentan realizar inmediatamente la busqueda arreglada, almacenan estas buscasdas erroneas para su uso posterior en caso de que otro usualrio haga algo similar.

Minimas reglas que deberian aplicarse:

1. Sacar blancos de comienzo y final(trim) - Pero no es suficiente, podria haber blancos entre medio
2. Buscar pasando todo a mayuscula o minuscula
3. Si se conocen abreviaturas, usarlas - BA por Buenos Aires
4. Los simbolos de punctuacion, eliminarlos
5. Si se conocen sinonimos, usarlos

Reglas especificas para un determinado tipo de datos

Este tipo de cosas se tienen que tener en cuenta:

- Fechas y sus formatos
- La hora y sus formatos
- Numeros
- Numeros decimales
- Strings correspondientes a nombres y apellidos (depende del orden)

Algoritmos busqueda aproximada

Soundex

Es un algoritmo fonetico, es decir, codifica una palabra segun como suena. Intenta solucionar problemas de pronunciacion, fue creado para el alfabeta ingles, pero existen implementacinoes para otros idiomas. Primer algorimo fonetico inventado, de la decada de 1930 y fue utilizado hasta la decada de 1990.

Idea, aquellas palabras que suenen igual, aunque no se escriban igual, deben ser codificadas de la misma manera. Lo que hicieron fue agrupar letras que sonaban parecido.

Soundex siempre devuelve un `OUT` de exactamente 4 caracteres formados por:

```
[Letra][Digito][Digito][Digito]
```

26 Letras	Pesos fonéticos
A, E, I, O, U, Y, W, H	0 -- no se codifica
B, F, P, V	1
C, G, J, K, Q, S, X, Z	2
D, T	3
L	4
M, N	5
R	6

Figure 2: Categorizacion de letras del algorimo Soundex

Pasos

1. Pasar todo a mayuscula y dejar solo las letras(digitos, simbolos de punctuacion, espacio, etc., se eliminan)
2. Colocar `OUT[0] = IN[0]` - La primera se la quedan pues es la mas importante
3. Se calcula la variable `last` como el peso fonetico de `IN[0]` 3.1 Calcular variable `current` con un peso fonetico de `iter`. Si es diferente a 0 y no concide con `last`. 3.2 `last = current`
4. Para cada letra en `IN`, iterar hasta completar 3 digitos o terminar de procesar `IN`.

Un caso, si un producto no se encuentra en la categoria esperada

Como usarlo Soundex no es una metrica, hay que definir como obtener una metrica a partir de Soundex.

Definicion de similitud Comparo caracter a caracter y cuento cuantas veces coicidieron. Si ninguno concide, entonces da 0. Si coinciden 3 caracteres, entonces da 0.75, es decir, la metrica va de 0 a 1.

> **Nota**: Un Hashmap es la mejor opcion para hacer un lookup, pero para que sea O(1) tiene que cumplir con ondiciones especiales. Para el caso de Soundex, un hashmap encubierto.

Metaphone

Aparece en 1990 y presenta mejoras en relacion a Soundex, no hace la clasificacion, sino que la salida son letras que representan los sonidos. Genera un encoding de longitud arbitraria. En wikipedia esta bien documentado el algoritmo, pero recomiendad usar la version en ingles pues es la que esta mejor explicada.

> Se recomienda ver la implementacion en `apache/commons-codec` una vez implementaste tu version, ahi hay muchos algoritmos conocidos.

Definicion de similitud La similitud entre 2 textos puede pensarse como la proporcion de caracteres coincidentes entre los encodings respecto de la misma longitud, pero cuando no tienen la misma longitud, no es trivial. Para ello se utiliza **Levenshtein Distance**.

Levenshtein Distance

No es un encoding, calcula la **minima** cantidad de operaciones necesarias para transformar un string en otro. Las operaciones validas son:

- Insertar
- Borrar
- Sustituir un caracter

> **Nota**: Esto es la version canonina, hay alternativas con mas operaciones.

Aquellos strings iguales deben tener distancia 0. Basicamente hay muchas maneras de ir a un string a otro, como podemos obtener la manera con la menor cantidad de pasos?

La forma manual de hacerla es con una matriz, arriba va una palabra y al costado otra, cada palabra debe comenzar con un meta-caracter pues debe ser un simbolo que no esa en el lenguaje.

	meta-c	B	I	G		D	A	T	A
meta-c	0	1	2	3	4	5	6	7	8
B	1								
I	2				x				
G	3								
D	4								
A	5								
T	6								
A	7								

Para calcular el valor de x, esa celda representa `L('BIG', 'BI')`. Pero hay varias posibilidades para decidir el valor de x en base a los anteriores, el objetivo es obtener el menor. Los tres valores candidatos:

1. `L('BI', 'B')`(Celda anterior, diagonalmente) + `G == I?0:1`
2. `L('BIG',' B')`(Celda anterior, verticalmente) + 1
3. `L('BI',' BI')`(Celda anterior, horizontalmente) + 1

La ultima celda sera el resultado.

Ejempo casi-resuelto:

	meta-c	B	I	G		D	A	T	A
meta-c	0	1	2	3	4	5	6	7	8
B	1	0	1	2	3	4	5	6	7
I	2	1	0	1	2	3	4	5	6
G	3	2	1	0	1	2	3	4	5
D	4	3	2	1	1	1	2	3	4
A	5	4	3	2	2	2	1	2	3
T	6	5	4	3	3	3	2	1	2
A	7	6	5	4	4	4	3	2	**1**

Este algoritmo es caracteristico, pues el minimo tomado en algun lugar intermedio llega al minimo global. Minimo local coincide con minimo global(no siempre ocurre).

Normalizacion La distancia es un numero cualquiera, si quiero un numero entre 0 y 1, se obtiene de:

$$LevenshteinNormalize(str1, str2) = 1 - \frac{Levenshtein(str1, str2)}{\max(str1.len, str2.len)}$$

Propiedades matematicas El algoritmo cumple con las siguientes propiedades.

- **Simetria**: $L(s1, s2) = L(s2, s1)$
- **Desigualdad**: $L(s1, s2) + L(s2, s3) >= L(s1, s3)$

Programacion dinamica Es una tecnica que consiste en rausar valores previamente calculados para no tener que recalcularlos repetidamente. Sirve, si para cierto calcula, pueden reusarce valores previos y tiene la desventaja de que estoy cambiando tiempo por espacio(`cache`).

Para buscar estos valores necesitamos una estructura de datos que permita buscar valores lo mas rapido posible -> matriz, vector, etc.

De esta manera, por ejemplo, el algoritmo `fibonacci` pasa de ser $O(2^N)$ a $O(N)$.

Variantes Otras variantes no consideran que las operaciones valos todas igual. Algunas es mas cara que otra y cambia la formula de distancia. Algunas agregan operaciones.

Ejemplo: Se agrega la operacion transposicion. Contempla el tipico error de escritura en el cual se invierten las letras.

Complejidad temporal Notar que a pesar de ser dos `if` anidados, la complejidad no sera $O(N^2)$ pues la matriz no sera cuadrada, tendra una complejidad de $O(N \cdot M)$.

Complejidad espacial Similar a la temporal ya que mantenemos la matriz, sera un orden de $O(N \cdot M)$

Optimizacion - Version compacta

Mas util si solo necesito conocer la similitud. Notar que no necesitamos todas las filas anteriores para seguir con la siguiente fila, entonces se propone un cambio de punteros, basicamente utilizando solo dos filas en vez de la matriz.

Solo hay que hacer un cambio de variables, no conviene copiar todo un arreglo. Notar, que tendra otra complejidad espacial. De todas formas, los dos algoritmos sirven, para distintas cosas. El menos compacto sirve para obtener los pasos que tendria que hacer para transformar los strings(haciendo el camino inverso).

Q-Grams(o N-Grams)

Consiste en generar los pedazos que componen un string. La distancia entre 2 strings estara dada por la cantidad de componentes que tengan en comun. No es un econding sino que simplementte realiza la comparacion.

Si Q es 1 se generan componentes de longitud 1. Si Q es 2 se generan **bi-gramas**, si Q es 3 se generan **tri-gramas**. Nosotros vamos a completar con $Q - 1$ al comienzo y al final para que todas aparezcan la misma contidad de veces.

Por ejemplo: `Q-grams(JOHN) = {J, O, H, N, #J, JO, OH, HN, N#, ##J, #JO, JOH, OHN, HN#, N##}`

String matching Para comparar dos strings, se compararn los arreglos de dos palabras y se cuenta cuantos elementos tienen en comun. Luego, la distancia sera la cantidad de elementos en comun. La implementacion que es mas usada, es tri-grama y esa es la que vamos a usar nosottros. Si al comparar aparece el mismo patron dos veces en uno y una vez en el otro, solo se cuenta una vez, es decir, la comparacion es 1 en 1.

Normalizacion

$$Q - grams(str1, str2) = \frac{\#TG(str1) + \#TG(str2) - \#TGNoShared(str1, str2)}{\#TG(st1) + \#TG(str2)}$$

Nota: En general, nuca se usa un algoritmo individual, sino que se usa una combinacion de varios. Utilizando una formula que los combine.

Algoritmos para busqueda exacta

Fuerza bruta o Naive

```java
public static int indexOf(String str, String pattern) {

    char[] s = str.toCharArray();
    char[] p = pattern.toCharArray();

    int j = 0;

    for (int i = 0; i < s.length; i++) {
        if (s[i] == p[j]) {
            j++;
        } else {
            i -= j;
            j = 0;  // Puede estar mal, pues tengo que ir para atras de nuevo
        }

        if (j == p.length) {
            return i - j + 1;
        }
    }
    return -1;
}
```

Voy comparando char a char y verificando si coinciden, recorriendo todo el array. El peor caso es que el query no encuentre el target.

- **Complejidad temporal** = O(n*m), donde `n = target.length` y `m = query.length`
- **Complejidad espacial**: O(1)

Este algoritmo no es el mejor ya que no aprovecha lo que aprendio en el recorrrido.

Knuth-Morris-Pratt

Examina el **target** de izquierda a derehca, pero usa conocimiento sobre los caracteres comparados antes de determinar la proxima posicion del patron a usar. Preprocesa el query antes de la busqueda una vez, con el objetivo de analizar la estructura(las caracteristicas del patrton query). Para ello contruye una tabla **Next** del mismo `length` que el query. No vuelve para atras, eso es muy importante ya que reduce la complejidad temporal. La tabla tiene en cada posicion `i` la longitud del **borde propio** mas grande para el substring desde 0 hasta `i`.

Implementacion de la computacion de Next:

```
private static int[] nextComputation1(char[] query) {
    int[] next = new int[query.length];

    int border=0;  // Length of the current border

    int rec=1;
    while(rec < query.length){
        if(query[rec]!=query[border]){
           if(border!=0)
                border=next[border-1];
           else
                next[rec++]=0;
       }
       else{
            border++;
            next[rec]=border;
            rec++;
        }
    }
    return next;
}
```

Es muy astuta la implementacion y tiene complejidad temporal O(n).

Para calcular el `search` sin el **backtracking**, avanza entre la tabla del `query` y el `target`, cuando no haya coincidencias se **shifttea** el `query` de la siguiente manera:

Supongamos que un `rec` que apunta al caracter en `target` y que `pqueary` que apunta a un caracter en `query`. Mientras haya coincidencia, avanzo en ambos. Cuando no la haya se **shiftea** query a `next[pquery-1]`, salvo que `pquery` sea 0, en cuyo caso hay que avanzar `rec` en `target`.

Finalmente, la complejidad temporal sera O(n+m)

Fulltext Retrieval

El problema es que KMP realiza la busqueda en un documento, pero si quisiera buscar sobre un grupo de documentos, nos conviene generar un indice de dicha coleccion para luego buscar a traves del indice. Lo que ocurre es que generalmente la **query** no siempre es la misma, pero los documentos no cambian, entonces conviene preprocesar los documentos(similar a lo que hace google). Esto se conoce como contruir el **corpus**. Eso es lo que hacen las bibliotecas de **Fulltext Retrieval**. Existe una biblioteca de codigo abierto para esto: **Lucene**. Tomamos ventaja de los indices obtenigos.

Basicamente, esta biblioteca nos permmite armar nuestro propio "search engine", tiene hasta el manejo de sinonimos.

Indices

Un indice es una estrucutra que permite llegar rapidamente al dato buscado, pero si se modifica un documento en la coleccion debe actualizarse. Es decir, su ventaja esta en la busqueda y tiene la desventaja de nececsitar actualizar la estructura cada vez que se modifique un documento.

Lucene usa un **archivo invertido** (conjnuto de terminos que dicen a que documentos pertenecen). Es un mapping:

Termino -> Documento

Por ejemplo:

```
MapReduce -> {Doc1, Doc2}
Grid -> {Doc2}
...
```

De esa manera evita tener que ir a mirar los documentos.

Definicion: Documento

Un documento en Lucene es una secuencia de **campos**(fields). Cuando se ingresa un documento en Lucene, automaticamente se asociara un ID(docid).

Definicion: Campo(Field)

Un campo en Lucene es un par nombre y secuencia de 1 o mas terminos.

```
// Como crearlo
FieldType aField = new FieldType();

// Como decidir se se almacena o indexa
aField.setStored(true);

// Como especificar como indexar(depende cuanto espacio queres que ocupe)
aField.setIndexOptions(IndexOptions.NONE);
// - NONE(no se indexa)
// - DOCS(se indiza solo con los doc ids en los que esta)
// - DOCS_AND_FREQS(similar a DOCS, y ademas se especifica la frecuencia)
// - DOCS_AND_FREQ_AND_POSITIONS(Idem a anterior, ademas de las posiciones ordinales)
// - DOCS_AND_FREQS_AND_POSITIONS_AND_OFFSETS(No lo vamos a usar, solo para frontend)
```

Para evitar escribir `DOCS_AND_FREQ_AND_POSITIONS` todo el tiempo, se usa `TextField`, el cual es simplemente un alias. La opcion de almacenar solo se podra hacer si no viene de un **stream**, sino lo estaria almacenando 3 veces: en el stream(file system), en el indice y ademas otra vez literal fuera del archivo.

Definicion: Termino(Term)

Un termino Lucene es una secuencia de bytes(Podrian interpretarse como Strings, numeros, etc. asociada a cierto campo. Dos secuencias de bytes con igual contenido pero asociadas a 2 campor diferentes se consideran diferentes.

Nota: Nosotros solo vamos a usar Strings como terminos.

Almacenamiento

Lucene es peculiar ya que tiene una zona donde guarda informacion que no es el archivo invertido. Al crear un campo(nombre y terminos) y asociarlo a cirto documento puedo optar por:

- Almacenarlo en Lucene pero fuera del archivo invertido. Eso significa que se lo almacena literal(sin procesamiento, no se lo separa en tokans, no se pasa a minuscula, etc.). Es decir, esta en Lucene pero no esta indexado, entonces no participa de las busquedas.
- Indexarlo en el archivo invertido Lucene. Eso significa que esta tokenizado o no(segun lo configure) forma parte del archivo invertido y sus terminos(to kens), entonces participan de las busquedas.

  Nota: Uno busca palabras especificas, no busca signos de punctuacion, preposiciones, etc. Solo nos interesan las palabras clave.

Un field puede ser solo almacenable, solo indexable o ambas cosas. Es obligatoria que sea, por lo menos, una de ellas. La informacion se almacena de distintas maneras dependiendo de la opcion que eleji para los fields. Es una tabla ordenada alfabeticamente por token.

Value term	Freq en docs	[docid:freqs in docif:[position]]
...	...	...

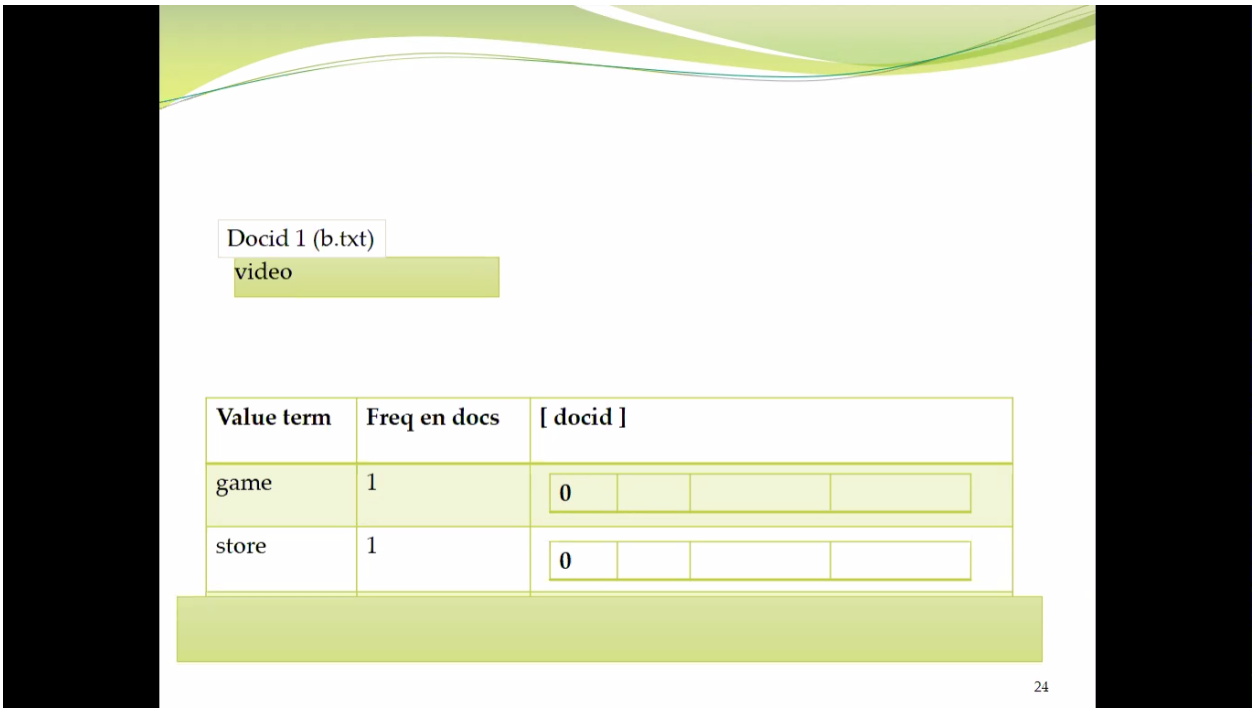

Figure 3: Ejemplo almacenamiento Lacene

Como se le asigna el id al doc? Si quiero el nombre y no el id? Para eso sirve el almacenamiento fuera del index, el nombre del path lo puedo almacenar en el almacenamiento aparte. Es un exelente uso para el almacenamiento fuera del indice en Lucene.

Ranking de documentos

Si busco un **query** y trae varios documentos, cual documento aparece primero? cual es mas relevante que otro? Esto se decide a partir de una formula.

$$\text{Score}(DOC_i, query) = \text{FormulaLocal}(DOC_i, query) \cdot \text{FormulaGlobal}(DOC, query)$$

Formula local: Quiere calcular que tan relevante es esa query respecto a un documento en particular a rankear. Por lo tanto, si busco un paper en Java, me va a importar mas el que mencione mas dicho lenguaje. Si dos documentos tienen la misma cantidad de apariciones, se elije el de mayor volumen.

$$\text{FormulaLocal}(DOC_i, query) = \sqrt{\frac{\#\text{frec term en } DOC_i}{\#\text{term existentes en } DOC_i}}$$

Formula global: Quiere calcular que tan relevante es esa query respecto a la coleccion de documentos existente. Tampoco se quiere linealidad, sino que tiene escala logaritmica.

$$\text{FormulaGlobal}(DOC, query) = 1 + \log_e(\frac{1 + \#\text{docs en la coleccion}}{1 + \#\text{docs que contienen la query}})$$

Multi-Query Si es una query multi termino, el score queda:

$$\text{Score}(DOC_i, query) = \Sigma_{\text{term in query, without NOT}}\text{FormulaLocal}(DOC_i, term) \cdot \text{FormulaGlobal}(DOC, term)$$

Si es con `AND` es solo los documentos con los dos cumplidos, pero se siguen sumando los scores.:3

Queries

Vamos a usar dos aplicaciones: IndexBuilder y TheSearcher(se encarga de aceptar consultas, utiliza el indice, solo devuelve los ids). **SOLO** usar las versiones de las teoricas, sino no nos va a funcionar porque cambio la API.

Hay dos maneras para hacer **queries**(Una mas alto nivel que otra):

- **API**
  - `TermQuery`
  - `PrefixQuery`
  - `PhraseQuery`
  - `WildcardQuery`
  - `FuzzyQuery` <- Usa Damerau-Levenstein(Con transposicion) con un MaxEdit=2
  - `BooleanQuery`
  - etc
- **Lenguaje lado cliente**
  - `TermQuery`: Con `content:game`.
  - `PrefixQuery`: Con `content:ga*`.
  - `TermRangeQuery`: Con `content:{gaming TO gum]`, donde `{` es abierto y `]` cerrado.
  - `PraseQuery`: Con `content:\"store game\"`, busca frase donde la primer palabra sea ‘store’ y la segundo ‘game’.

- `WildcardQuery`: Con `content:g??e`.
- `FuzzyQuery`: Con `content:gno~2`, no se toman mas de 2, estaria muy lejos.
- etc.

El `QueryParser` es una extension que permite simplificar los query. Ejemplo de uso:

```
String queryStr= "content:game";

QueryParser queryparser = new QueryParser(null, new StandardAnalyzer() );
Query query= queryparser.parse(queryStr);
```

Si busco 'ga' con `content:ga` no va a encontrar 'game', pero si quiero buscar el analogo a `PrefixQuery` busco prefijo `ga` con `content:ga*`. Si ya se que las queries siempre van a ser en `content`, se podria pasar como default y no tener que poner el prefijo.

```
String queryStr= "game";

QueryParser queryparser = new QueryParser("content", new StandardAnalyzer() );
Query query= queryparser.parse(queryStr);
```

> **Nota**: Si pones `content:store content:game` lo va a tomar como `content:store OR content:game`, hay que tener cuidado. Tambien acepta `||` como **or**.

Analyzers

Si buscamos 'Game' no lo va a encontrar, esto lo solucionamos con un analyzer, el cual se va a encargar de pasar a minuscula, sacar numeros, etc. Permite separar los tokens, en el ejemplo anterior usamos el `StandardAnalyzer`, pero hay muchos mas, permitionme filtrar con precision lo que quiero que se indexe y lo que no. Si ninguno te sirve, podes usar un `CustomAnalyzer` y personalizarlo como quieras.

```
Analyzer a = CustomAnalyzer.builder()
            .withTokenizer("standard")
            .addTokenFilter("lowercase")
            .addTokenFilter("stop")
            .addTokenFilter("porterstem") // Word family
            .addTokenFilter("capitalization")
            .build();
```

El `SpanishAnalyzer` busca la raiz de la palabra ya que podria ser similar al del castellano(Stemmer Algorithm).

Solr y Elasticsearch

Lucene no ofrece escabilidad, para ello se utiliza Solr o Elasticsearch. Para realizar bsuquedas de gran escala, se ditribuye la busqueda en varias computadoras. Una compuradora dirije y pide la busqueda y el resto realiza la busqueda en diferentes documentos.

Unidad 3: Estructuras lineales

Arreglos

Indices y estructuras posibles

En la seccion anterior se utiliza un indice como estructura auxiliar para guardar los terminos. Es muy importante poder realizar busquedas de la manera mas eficiente posible. Se utiliza una clave de busqueda que permite encontrar rapidamente al resto de la informacion(como un mapa). Por ejemplo, puedo buscar la informacion de un aluno por legajo, por edad, etc. Para el caso de la edad, el indice esta **descompactado** ya que aparece el 20 mas de una vez. Si el indice esta **compactado**, aparece una edad una unica vez con una lista asociada de los alumnos con dicha edad.

Operaciones Ahora bien, el indice no solo se utiliza para buscar, sino que tambien hay operaciones:

- Busqueda
- Insercion
- Borrado

La complejidad temporal de cada una depende del tipo de estructura, por ejemplo, si se trata de un arregla con espacio prealocado, si quiero insertar es simplemente cuestion de insertar al final, pero todo cambia si tiene que ser ordenado o tengo que aumentar el espacio.

No hay estruturas de datos perfectas, dependen del caso. Nosotros vamos a utilizar un arreglo ordenado,

Para el caso de la **insercion** es muy constoso tener que alocar memoria en caso de requerir expandir el arreglo, por lo tanto, es muy malo ir alocando de a uno. Es una practica comun hacer crecer el arreglo de a chunks. Para insertar en el medio del arreglo, hay que ir swapeando del final hacia adelante.

Por otro lado, en caso del **borrado** tambien deberia ser de a chunks ya que no es ideal tener mucho espacio vacio, pero comunmente no se hace. Si queremos hacerlo podemos, pero no va a estar mal no hacerlo.

Teorema maestro

Permite calcular la complejidad temporal de un algoritmo recursivo, solo sirve para recurrencias simetricas. Si formulando la cantidad de operaciones de un algoritmo recursiva me queda como:

$$T(N) = a \cdot T(\frac{N}{b}) + c \cdot N^d$$

Entonces estamos en condiciones de utilizar el teorema maestro.

Donde:

- N es el tamano de entrada del problema
- $a \in \mathrm{N}_{\geq 1}$ es la cantidad de invocaciones recursivas realiza ese paso
- $b \in \mathrm{N}_{>1}$ es la tasa en la que se reduce el tamano del input
- $c \in \mathrm{R}_{>0}$
- $d \in \mathrm{R}_{\geq 0}$

Casos

- Si $a < b^d$ entonces el algoritmo es $O(N^d)$

- Si $a = b^d$ entonces el algoritmo es $O(N^d \cdot \log(N))$
- Si $a > b^d$ entonces el algoritmo es $O(N^{\log_b(a)})$

  Nota: No siempre se puede aplicar el teorema, como el caso de Fibonacci

Expansion recursiva

Otra manera de encontrar la complejidad de los algoritmos recursivas, lo importante es encontrar Times(N), si esta mal despues todo va a estar mal.

Complejidad para version iterativa y para recursiva

Ya calculamos la complejidad para busqueda binaria implementada como recurrencia, pero si la implementamos de manera iterativa, cambia la complejidad?

De manera iterativa, solo voy ajustando las variables, mientras que de manera recursiva vuelvo a llamar a la misma funcion. Pero **no** cambia la complejidad temporal, por lo tanto yo puedo elejir el metodo que mas me guste.

Sin embargo, la coplejidad espacial si cambia, la complejidad espacial de la recursiva es $O(\log_2(N))$ mientras que la iterativa es $O(1)$.

Metodos de ordenamiento

Quicksort Opera **in-palce**, es decir opera sobre el mismo arreglo, aplica tecnica **Divide&Conquer** y puede implementarse recursivamente o iterativamente.

Particiona en subarreglos:

- En cada subarreglo elige un pivot y ordena para que todos los elementos a la izquierda del pivot sean menores que el y los de la derecha sean mayores que el. Comeinza con el primer elemento, luego toma el primer elemento a la derecha del lugar donde quedo el primer elemento del arreglo.
- Si un subarreglo tiene 0 o 1 elemento, esta ya ordenado(no continua) y termina la recurrencia.

Complejidad temporal En el peor caso no se puede aplicar teorema maestro, el peor caso es cuando el arreglo viene ordenado(acendente o decendentemente). Esto se debe a que no se parte a la mitad.

Entonces, usamos otra forma para calcular la complejidad temporal para el peor de los casos.

$$Times(N) = N + Times(N-1)$$

Por lo tanto, sera $O(N^2)$.

Para el resto de los casos, se divide a la mitad cada vex, entonces:

$$Times(N) = Times(\frac{N}{2}) + \frac{N}{2}$$

Sera $O(N \cdot \log(N))$

Notas que en el mejor caso si se puede aplicar el teorema maestro y se llega al mismo resultado.

Variantes Como mejorar quicksort para que cuendo venga casi ordenado no de una complejidad temporal tan mala.

Mergesort Hay dos versiones, una facil que genera un arreglo paralelo y otra dificil que es **in situ**(no crea otro arreglo). Funciona de la siguiente manera:

1. Divide el arreglo en dos mitades recursivamente hasta que no se pueda dividir mas
2. Recursivamente ordenar cada mitad
3. Unir las dos mitades

Coplejidad temporal Primero planteo el numero de ecuaciones:

$$Times(N) = 5 + (N-1) \cdot 4 + 2 \cdot Times(\frac{N}{2})$$

Veo que tiene la forma del teorema maestro, entonces:

$$Times(N) = N \cdot 4 + 2 \cdot Times(\frac{N}{2})$$

Sera $O(N \cdot \log(N))$

Generics en Java

Java es un lenguaje estaticamente tipado, hay que decir que tipo es una variable antes de usarla. Sin generics, son casteos son una posibilidad de errores que se detectan en tiempo de ejecucion. Generics esta pensado para parametrizar y minimizar errores.

Tecnicamente hablando, Generics fue implementado usando la **Tecnica de Erasure**, la cual consiste en reemplazar todo tipo de parametro con su "bound/restriccion" y si no lo hay reemplaza por Objects.

Por ejemplo:

- `<T> is unbound => Object`
- `<T implements Comparable> => Comparable`

`private T[] arreglo = new T[];` falla, no compila. Entonces, `ArrayList` internamente implementa un arreglo, pero para Generics no se puede simplemente declarar un arreglo, como lo hace? No puedo crear dinamicamente un arreglo de tipo parametrico(en tiempo de ejecucion) porque su tipo no se conoce ya qye en compilacion se hizo erasure. Hay tres maneras que podriamos usar para solucionar esto, pero la tercera no nos sirve en nuestro caso:

1. Guardar un arreglo de `Objects`(no T). Castear cuando sea necesario, entonces cuando me piden un elemento simplemente uso el casteo(seguro porque ya se el tipo).
2. Usar reflection, internamente uso un arreglo de T, pero el new tiene que usar reflection. Por lo tanto, en `initialize` pido que me de `Class<t> the class` con un `@SuppressWarnings("unchecked")`. Usando `(T[]) Array.newInstance(theCLass, dim)`.
3. Usando `(E[]) new Object[dim]` (con el `SuppressWarnings`), parece similar al reflection, pero en realidad estoy con un arreglo de `Object` que tengo que castear cada vez que necesite un elemento, mientras que reflection tiene un arreglo de T.

Vamos a descartar la tercera, pues no permite tener un Generics que sea **bounded**. Por lo tanto, vamos a usar la primera o la segunda.

Como incluir `.jar`

Primero, dejar la biblioteca que quieras agregar en un path sin blancos. Luego, en Maven:

```
<!-- Sin Reflection -->
<dependency>
    <groupId>ar.edu.itba.eda</groupId>
    <artifactId>IndexWithArray</artifactId>
    <version>1</version>
    <scope>system</scope>
    <systemPath>/.../IndexWithArray-1.jar</systemPath>
</dependency>
```

Stack

En esta estructura nos enfocamos en el orden de llegada de los elementos. Por ejemplo, los editores de texto implementan este tipo de estructura para "deshacer" acciones. Las operaciones que ofrece son:

- `push`
- `pop`
- `isEmpty`
- `peek`, no obligatorio, pero muy util.

Implementacion

Con una simple lista encadenada o un arreglo se puede implemeentar. En caso de arreglo la contiguedad esta garanttizada y nunca hara falta mover elementos, pero si se acaba el espacio se debe buscar espacio conttiguo en otro lugar. Por otro lado, si se uttiliza una lista lineal simplementte ecadenada, el elemento mas a la "izquierda" sera el tope. Java ya cuenta con una implementacion para el Stack, pero no es muy buena ya que al ectender otra clase, tiene un monton de metodos innecesarios. Es decir, desde el punto de vista OOP, es una mal practica.

Se nos permite usar la implementacion de Java, pero es recomendable que hagamos nuestra propia version. Podemos usar la de Java, pero solo con los metodos mencionados anteriormente, por ejemplo, no podemos usar el `add`.

Caso de uso: Evaluador de expresiones

En este caso solo vamos a considerar los operadores binarios. Las expresiones se pueden clasificar la la siguiente manera: prefija(operador antes de operandos), infija(operador enntre operandos) y postfija(Operados despues de operandos). Un problema con la infija es que hay operadores con mayor procedencia, tambien se pueden operar de izquierda a derecha, pero en casos como la potencia es de derecha a izquierda, hay que tener en cuenta esto. Tambien, exite la polaca, infija reversa, etc. pero solo nos enfocamos en las tres de antes.

Hay varios algoritmos para evaluar expresiones, el siguiente algoritmo toma expresiones posfijas:

1. Cada operador es una expresion posfija se refiere a los operandos previos a la misma.
2. Cuando aparece un operando hay que postergarlo porque no se puede hacer nada con el hasta que no llegue el operador, se pushea al stack.
3. Cuando aparezca un operador en la expresion implica que llego el momento de aplicarselo a los operandos, se procede a popearlos del stack.
4. Cuando se termine de analizar la expresion de entrado el resulltado de su evaluacion es el unico valor que debe quedar en la pila.

   Nota: La clase `Scanner` de Java es muy util para este tipo de aplicaciones pues permite recibir un input por archivo, recivir un input por entrada estandar, separalo en toquens. Hay que tener cuidados con los metalenguaje, como con el +, hay que escparlos con \\.

Infija a posfija

Existe un algorimo muy famoso para realizar esta operacion. La idea es que ada vez que aparezcan varios operadores se consultara una tabla que indique cual se evalua primero. Si dos operadores tienen la misma precedencia, se utiliza la regla de asociatividad para saber cual se evalua primero.
Para comparar, vamos a usar una tabla donde en la primer columna va el operador previo y en la primera fila el operador actual. Entonces, si en fila hay un * y en columna un +, entonces se devuelve un `false` pues el que estaba en tope de la fila tiene menos precedencia sobre el elemento actual.

Pasos del algoritmo:

1. Si la pila esta vacia, se pushea el operador **current** ya que no se lo puede comparar.
2. Si la pila no esta vacia:
   - Si el tope de la pila tiene mayor precedencia que el current, entonces se realiza un pop.
   - Si el tope tiene menor precedencia, el current no se puede ir todavia.
3. Al finalizar, se popea lo que haya quedado en el stack.

Esto funciona para los operadores asociativos a izquierda, pero ahora si queremos implementar el operador exponencial necesitamos considerar el asociativo a izquierda, pero no hace falta cambiar el algoritmo, simplemente basta con modificar la tabla. Es un buen disano debido a su flexibilidad.

	+	-	*	/	^
+	true	true	false	false	false
-	true	true	false	false	false
*	true	true	true	true	false
/	true	true	true	true	false
^	true	true	true	true	false

Par agregar soporte para parentesis, hay que considerar lo siguiente:

1. Si se encuentra un parentesis que abre, debe postergarse hasta encontrar uno cerrado.
2. Si el operador current es un parentesis que cierra el mismo debe sacar todos los operadores del stack hasta encontrar el parentesis que abre. Luego, se sca el parentesis que abre de la pila y de lo descarta(no hay parentesis en postfija.

Para incorporar esto vamos a usar la tabla. Notar que no tiene sentido que en el stack haya un parentesis que cierra.

	+	-	*	/	^	(	)
+	true	true	false	false	false	false	true
-	true	true	false	false	false	false	true
*	true	true	true	true	false	false	true
/	true	true	true	true	false	false	true
^	true	true	true	true	false	false	true
(	false	false	false	false	false	false	false

Cola

Son muy utilizadas para algoritmos, por ejemplo, BFS en grafos. O simplemente se usan para manejar una fila the eventos como el caso de un **pipe**.

Definicion: Queue

Coleccion de datos ordenada por orden de llegada. La unica forma de acceso es por medio de dos elmentos distinguidos: FIRST(El mas antiguo) y LAST. Las operaciones que ofrece son:

- `queue(element)`
- `deque()`
- `peek()`
- `isEmpty()`
- `size()`, esta es opcional, pero puede ser util.

Si se implementa como lista, conviene que el `first` sea el primer elemento y `last` el utlimo. Esto permite O(1) para obtener `first`.

En el caso de implementarlo com arreglo, ambas maneras tienen un problema, tengo que mover todo si `last` es el primero y si `first` es el primero va a haber espacios en blanco cada vez que haga un `deque()`. Si se acaba el espacio, hay que realocar(conocido como **unbounded queue**), pero existe una forma de arreglar esto: Utilizando un arreglo circular. Si no el arreglo no va a crecer, entonces se conoce como **bounded queue**, este seria el caso del queue circular.

En conclusion, si se tiene un unbounded queue la mejor implementacion es la que utiliza una lista, pero si tengo un bounded queue la mejor implementacion es la qu utiliza un arreglo.

Lista

Supera el problema de contiguedad y re-alocacion de espacio, podemos pensar en estructuras de datos que permitan que los elementos esten "fisicamente aislados y logicamente conectados". Sin embargo, nunca superara una lista ordenada al arreglo ordenado.

Lista lineal simplemente encadenada

Es una estructura de datos que esta compuesta por 0 o mas nodos. Cada nodo(elemento) tiene referencia al siguiente.

Insertar

Hay tres maneras de insertar un nodo, tenemos que saber y entender las 3:

1. Iterativa desde afuera de nodo.
2. Recursiva desde afuera de nodo.
3. El nodo mismo debe tener su propia funcion insert. Se conoce como **delegacion**.

Iterador

Vemos que la lista no presenta ventajas en comparacion con un arreglo pues la complejidad de la busqueda, insercino y borrado es O(N). Hay un caso especial que seria el del iterador, ya que utilizando el iterador y quiero eliminar un elemento, al ya estar apuntadno al elemento, las operaciones seran O(1).

Siguiendo las especificaciones de Java:

- `remove()` de iterador tiene que invocarse luego de un `next()`.
- No se pueden invocar 2 `remove()` seguidos(tiene que haber un `next()` en el medio).
- Si no se satisfacen esas condiciones de lanza una `IllegalStateException`.

Hay cosas que no vamos a poder validar y que haran la lista inconsistente(responsabilidad del programador, no nuestra):

- Tener dos iteradores anidados.
- Tener dos iteradores anidados y hacer el remove del segundo iterador y luego el del primer iterador(estas eliminando el elemento sobre el que estas iterando).

Unidad 4: Hashing

Es una alternativa para solucionar varios problemas, se utiliza para hacer lookup rapidos de alguna componente. Sin embargo, problemas que soluciona:

- Representar la clase `Bag`
- Representar la clase `Set`
- En algoritmos como fibonacci, se puede implemenar un cache

No sirve para obtener el mayor, el menor, un rango, un arreglo ordenado. Sirve para hacer busqueda puntual, toma la idea de arreglo y la aprovecha al maximo, por eso su imlementacion siempre tiene un arreglo. El problema es que el lugar que ocupa una componente no depende del orden de llegada, la posicion es fuertemente dependiente de los valores de las otras componentes.

Hash table

Estrucutra de datos que utiliza un arreglo(lookup table) para almacenar pares key/value de una forma muy especial. No mantiene contiguedad, ni orden de las componentes. Si Ω es el universo de los items que queremos almacenar y tenemos un arreglo Lookup subjacente, entonces, utiliza una funcion de hashing $hash : \Omega \rightarrow [0, |lookup| - 1]$, donde $|lookup|$ es la cantidad de ranuras.

Prioriza la busqueda, tratando de que el algoritmo tenga complejidad cercana a O(1) en la busqueda. Si para distintos elementos devuelve el mismo indie, hay que solucionarlo, eso lo veremos mas adelante.

Primer escenario

Suponemos que hay suficiente espacio, o sea $|\Omega| << |lookup|$. Sea Ω el conjunto de claves a hashear, y sea LookUp el arreglo para albergar los pares key/value. Asumimos $|\Omega| \leq |Lookup|$, es decir, hay posibilidad de que a cada key se le asigne una ranura de LookUp.

Como los keys que proporciona el usuario son tipos opacos(TAD, Objetos) es bueno solicitarle que proporcione una funcino de:

$prehash : \Omega \rightarrow N$ Eso es lo que nos hace implementar Java, son numeros naturales en general, no le vamos a pedir al usuario que acomode el numero a la cantidad de ranuras que nosotros reservamos.

Y nosotros, el que implementa el hashing, tenemos que garantizar que $hash : \Omega \rightarrow [0, |lookup| - 1]$ (cantidad de ranuras) porque la ranura tiene que ser valida, hacemos:

$$hash(key) = prehash(key)\%|Lookup|$$

Definicion: Funcion hash perfecta

Una funcion hash perfecta es dificil de encontrar, pero existe, donde no hay colisiones. Por ejemplo, en Soundex implementamos una tabla de hash perfecta pues era un hashing fijo. Por lo tanto, nuestro objetivo es crear una funcion hash que minimice las coliciones.

Definicion: Colision

Se dice que 2 claves key1 <> key2 **colisionan** si hash(key1) = hash(key2), es decir, se le asigna la misma ranura.

Factor de carga(current load factor)

Se define el factor de carga como:

$$CLF = \frac{|\text{Keys usadas}|}{|lookup|}$$

El algoritmos de insercion **provisorio**(porque todavia no manejamos colisiones) consiste en ir a la ranura correspondiente y proceder segun el caso:

1. Si esta ocupada por el mismo key $\Rightarrow$ Era un update. Lo actualiza.
2. Si esta ocupada por otra key $\Rightarrow$ Era insercion con colision. Esta insercion no es exitosa porque todavia no manejamos colisiones, lanzamos una excepcion.
3. Si esta vacia $\Rightarrow$ Era insercion exitosa. Primero insertar allo. Luego, chequear si el factor de carga supera un cierto umbral predefinido(**Load Factor Threshold**) y si eso ocurre se duplica el espacio y se rehashean todas las claves.

Entonces, la mayoria de los casos se va a comportar como orden 1, pero cada tanto va a haber que reordenar pues cada tanto se degrada. En realidad es O(1) en promedio, por lo que realmente es $\Omega(1)$.

Hashing sobre strings

- **Metodo 1**: ASCII del primer caracter

- **Metodo 2**: Suma de los ASCII, pero se le puede agregar un numero primo en la formula(como hace java), va a hashear mucho mejor, es decir, en la suma se le multiplica por un numero primo a cada valor asccii que se suma.

Funcinoes para claves numericas

- **Division o Modulo**: $hash(x) = x \cdot modm$ Donde m debe ser primo.
- **Mid-square**: Se calcula el cuadrado de un numero y se toman los bits centrales coo lugar donde hashear. Por ejemplo, si X = 14, X^2 = 23420 y hay que tomar los bits centrales para poder hashear. Si la tabla tubiera 11 elementos usaria el numero 34 % 11 (o 42 % 11).
- **Folding o Plegado**: Se divide el numero en zonas de la misma longitud. Se las suma y se toman los bytes necesarios. Por ejemplo, si X=20112241203123 y la tabla tiene 101 elementos, se arman grupos de a 3 digitos, se obtiene: 020 + 112 + 241 + 203 + 123 = 699 y se lo hashea a 699 % 101.
- **Analisis del Digito**: Implica un conocimiento de antemano de las caracteristicas de la poblacion. Se analiza los patrones de las claves, en busca de la informacion de la clave que menos se repite. Por ejemplo, si las claves para hashear los alumnos de una facultad en cierto ano fuera su DNNI, no convendria elegir suus primeros digitos porque todos los alumnos de un mismo ano comienzan con las mismas cifras de DNI.

Manejo de colisiones

Hay dos maneras de resolverlo, la mas complicada es la primera:

1. **Open Addressing or Closed Hashing**: Cuando hace la colision, busca un lugar disponible dentro de la tabla. Guarda los elementtos que colisionaros denttro de la misma tabla.
2. **Open hashing or Closed Addressing or Chaining**: Fuera del hashing se almacenan los elementos que colisionaros, pueden ser listas o arboles. Comienza a degradarse e orden 1.

Open Addressing

Cada ranura puede tener null(vacio o baja fisica). Aunque la ranura no este vacia puede ser que el elemento no este(Se haya borrado, pero no se lo quito), ya que hay que manejar el concepto de bajas logicas(ademos de las fisicas). Es decir, una ranura representa 3 estados. Formas de resolver:

Rehasheo Lineal(lineal probing) Si hay colision en la ranura i, entonces intentar con la ranura i+1, y asi siguiendo hasta encontrar que el elemento(se hace update) o encontrar un lugar vacio(baja fisica) y se la inserta alli. Con esta tecnique si hay lugar lo encuentra seguro. Este algorimo fue creado para disco, nosotros lo implementamos en RAM asi que los hacemos en RAM.

- **Borrado**: No se puede reemplazar al lugar borrado por una ranura vacia porque la busqueda de alguna clave puede necesitar "pasar sobre ella" si hubo colision. Se debe manejar dos tipos de borrado, fisico y logico. El borrado fisico se lo usa cuando la ranura que le sigue esta tambien borrado fisicamente, mientras que el borrado logico se lo usa en caso contrario, o sea cuando la ranura que le sigue esta ocupada o borrada logicamente.
- **Busqueda**: Se comienza buscando en la ranura calculada. Si el lugar esta con baja fisica seguro que no esta en otro lado. Caso contrario si esta amrcado como ocupado y coincide con el valor esperado, se ha encontrado.

- **Insercion**: Si la ranura calculada esta marcada como baja fisica, el elemento se inserta alli. Caso contrario hay que comenzar a navegar con las sucesivas.

  NOTA: La baja fisica afecta la performace, para no rebotar de mas en las colisiones, lo mejor en el algoritmo de insercion es no insertar en el primer espacio libre que se encuentra, sino en la primera baja logica que se encontro en el camino hasta descubrir que se podia insertar(se hallo baja fisica).

Rehasheo Cuadratico(quadratic probing) El intervalo entre ranuras a usar, si hubiera colision, sera cuadratica.

Unidad 5: Arboles binarios

Por ejemplo, se utilizan para compilar expresiones, como las matematicas. Se pueden usar estrategias para optimizar las expresiones a la hora de evaluarlas(re estructurar el BT), como sacar factor comun, por ejemplo. Se pueden utilizar para cualquier cosa que tenga una representacion jerarquica. Tambien, si el BT estuviera ordenado, podria usarse como soporte para indices.

Arboles de expresiones

Se utiliza para representar expresiones algebraicas. Los nodos internos representan operadores(binarios o unarios) y las hojas(nodo sin hijos) constantes(o variables). Hay distintas maneras de recorrer el arbol. Para la implementacion tenemos dos casos, si el token es un parentesis entonces se crea un nodo con dos hijos, y si es constante se le agrega una hoja a dicho nodo. Si tengo dos parentesis anidados, es decir, `((2+3))` dara error, pero cuando llegue al segundo cierre y se de cuenta de que no hay otro operando para aplicarle al primer parentesis.

Caracteristicas

- Permite representar expresiones en notacion infija(no esta ordenado por el contenido).
- Asi como usabamos una pila y una tabla de precedencia de operadores para pasar de una expresion en notacion infija a postfija(para eliminar ambiguedad) y luego con una pila evaluabamos la expresion, ahora tambien a partir de una expresion contruiremos el arbol de expresiones asociado y lo evaluaremos para devolver el valor de la expresion.

Aclaracion

Para evitar la discusion sobre la precedencia de operadores, vamos a aceptar solo expresiones infijas que tengan parentesis. Los operadoes son todos los binarios y es obligatorio que toda expresion tenga parentesis.. Para el input vamos a pedir que finalize en un `\n`.

Formalmente

Una expresion arismetica E esta dada por las siguientes reglas de derivacion:

- E $\rightarrow$ (E + E)
- E $\rightarrow$ (E - E)
- E $\rightarrow$ (E * E)

- E → (E / E)
- E → (E ^ E)
- E → cte

Es importante poder hacerlo de manera formal, hace que el proceso sea mucho mas ordenado a la hora de implementar algo.

> **Nota**: Hay una manera de testear outputs a consola, es la siguiente(Para ejemplos, ver testeos de ExpTree):
>
> ```
> System.setOut(System.out);
> ByteArrayOutputStream outContent = new ByteArrayOutputStream();
> System.setOut(new PrintStream(outContent));
>
> myExp.postorder();
> assertEquals( "15 3 + ", outContent.toString());
> ```

Arboles binarios en general

Si quisieramos guardar los datos en un arcihvo de texto, para luego recontruirlo en otro momento desde el archivo de texto, algun recorrido vendria bien. Que convenciones hay que tener en cuenta para generar un archivo de texto en el cual se guarde un arbol y que tmb se pueda reconstruir? Queremos poder:

- Recontruir leyendolo.
- Sin haber contruido nunca, editar un archivo, escribir la info con su estructura y leer.

Podria servir: preorder, postorder, inorder y por niveles. **Por niveles** es la mas facil de leer para las personas y mucho mas linda, la representacion coincide con el grafico(cosa que no se puede mantener con las otras opciones). Para pasar a texto, hay que colocal un *dummy*, simbolo para completar los espacios, vamos a usar un metalenguaje `?`(suponiendo que no se usan en los demas datos). Quedaria, por ejemplo:

```
    *
++     -10
  /
++     -10
 ?3.5
```

Pasa a:

```
*
++ -10
? / ? ?
? ? ++ -10 ? ? ? ?
? ? ? ? ? 3.5
```

Algoritmo

Tiene 3 acciones a contemplar

1. Genero raiz y postergo que hay que hacer con ella, hasta que llegue el token. Ni siquiera se si tendra cero, uno o dos hijos. Le asigno peso a cada nodo, para diferenciarlos cuando se resuelve esto en papel(primero es $20).
2. Saco pendiente y leo token. Completo dato, como no se si $20 tendra cero, uno o dos hijos(depende del token), pido que cuando llegue el momento se los proceso, los agrego al queue a los hijos.
3. Si me encuentro con signo de pregunta, tengo que consumirlo y avisar que tiene que consumir dos token, entonces agrego al queue dos nodos null.

Todo esto se realiza con un queue interno.

> **Nota**: En informatica, codigo y dato son intercambiables. Si es dato puedo pasarlo a accion y viceversa.

Definicion: Arbol binario completo(complete)

Un arbol binario es completo si todos sus niveles tienen todos los nodos posibles, excepto posiblemente el ultimo. El ultimo nivel tiene los nodos lo mas a la izquierda posible(completa de izquierda a derecha).] ### Definicion: Arbol binario lleno(full)

Un arbol binario esta lleno si todos los niveles tienen todos los nodos posibles. No es lo mismo que el completo necesariamente.

La estrategia de serializar a disco generico un arbol de que tipo respecto al concepto de completitud/lleno?

Es completo, lo garantiza. No necesariamente full al menos que hayamos especificado que complete hasta el final el ultimo nivel.

Definicion: Altura

Longitud(cantidad de ejes) del camino mas largo desde la raiz hacia las hojas. Un nodo formado por una raiz tiene altura 0.

Arboles de busqueda - BST

Los usos de los arboles son multiples, uno es manejar una lista ordenada. El BST es un árbol binario donde cada nodo no vacío cumple la siguiente condición: todos los datos de su subárbol izquierdo son menores o iguales que su dato, y todos los datos de su subárbol derecho son mayores que su dato.

Operaciones

- **Insertar**: Crece desde las hojas, si es mayor o igual que el nodo, se inserta como hoja a la derecha y si es menor se inserta como hoja a la izquierda. Empieza en root, si es mayor o igual va al subarbol derecho y viceversa, de esa manera con todos los nodos hasta llegar a una hoja. CHEQUEAR
- **Borrado**: Si el nodo a eliminar es una hoja, actualizar a quien lo apunta a el. Si el nodo a eliminar tiene un solo hijo, le entrego a el que lo apunta su unico hijo. Si el nodo a eliminar tiene dos hijos se procede en dos pasos: Primero se lo reemplaza por un nodo lexicograficamente adyacente(su predecesor

inorder, el mas grande de los nodos de su subarbol izquierdo, o bien su sucesor inorder o sea el mas chico de los nodos de su subarbol derecho), y finalmente se borra el nodo que lo reemplazo(sera facil de borrar porque no tendra dos hijos).

Nota: El projecto de BST lo vamos a estructurar de manera MVC(Model View Controller).

Nota: Java no me deja parametrizar el iterador, pero podria pasar el parametro anteriormente, por ejemplo, con un metodo `setTraversal()` anterior a llamar a `iterator()`.

Arboles balanceados

El BST presenta problemas, el peor caso es cuando esta totalmente desbalaneado, haciendo que operaciones como busqueda tengan una complejidad O(N). Lo que queremos es que tenga la menor altura posible, entonces hay que tratar de controlar la altura.

Algoritmo AVL

BST balanceado por altura, crado por rusos en 1962, la primer version de BST que se balancea dinamicamente. Un AVL es un BST donde en cada nodo la **diferencia**(Factor de balance) de alturas entre sus dos arboles es a los sumo 1.

Un AVL es un arbol con buena forma, las inserciones y borrados en un AVL estan definidas de tal manera que garantizan que se pueden realizar en O(log(N)) y garantizan ser **invariantes** ante su propiedad. Es un BST donde en cada nodo el factor de balance es a lo sumo 1.

Operaciones

- **Insersion**: Se inserta como BST, si esta desbalanceado de aplican rotaciones para que siga siendo AVL, se rota el arbol con pivote mas joven desbalanceado(mas cercano al nodo insertado), hay dos casos:
  - **Caso 1**: Si se inserta a la izquierda de un nodo con factor balance 1, ese nodo va a tener factor balance 2 y deja de ser AVL. Hay que rotar.
  - **Caso 2**: Si se inserta a la derecha de un nodo con factor de balance -1, ese nodo va a tener factor de balance -2 y deja de ser AVL. Hay que rotar.

  Nota: Para determinar el factor de balance se hace altura de arbol izquierdo menos altura del arbol derecho.

Consideraciones Como en inserciones/borrados tenemos que calcular la diferencia de altura entre los subarboles muy frecuentemente, conviene almacenar el valor en cada nodo. El peor caso de este tipo de arboles es el arbol de fibonacci, es el AVL que presenta el peor caso de balanceo.

Arbol de fibonacci Se define de la siguiente manera:

- Orden 0 es el arbol nulo.
- Orden 1 es un nodo
- Orden $h \geq 2$ es un arbol que tiene:
  - Como hijo izquierdo un fibonacci de orden $h - 1$

– Como hijo derecho un fibonacci de orden $h-2$

Un arbol fibo(6) tiene 20 nodos y una altura de 5, comparado con uno completo de igual cantidad de nodos como mucho hay una diferencia de altura(4 de altura). Por lo tanto:

$$AVL_h \text{ tendra 1 nodo } + \text{ cantidad nodos altura } h-1 + \text{ cant nodos altura } h-2$$

	AVL altura h con menor cant de nodos	Cant nodos en relación a los números de fibo?
	Altura 0 CantNodos=1	nrofibo(altura+3)-1 = 1
	Altura 1 CantNodos=2	nrofibo(altura+3)-1= 2
	Altura 2 CantNodos=1 + 1+ 2 = 4	nrofibo(altura+3)-1= 4
	Altura 3 CantNodos=1 + 4 +2 = 7	nrofibo(altura+3)-1= 7
	Altura 4 CantNodos=1 + 7 + 4 = 12	nrofibo(altura+3)-1= 12
	Altura 5 CantNodos=1 + 12 + 7 = 20	
	...	
	Altura h CantNodos=?	nrofibo(h+3)-1

Figure 4: Relacion de la altura con cantidad de nodos

Por lo tanto: $N(h) = \text{nrofibo}(h+3) - 1$. Esta sera la minima cantidad de nodos para un AVL de dicha altura.

Por otro lado, el numero de fibonacci puede aproximarce con en **golden number** $a = 1.618$, luefo la cantidad de nodos de un AVL de altura h sera(utilizando la formula anterior):

$$n \geq \frac{a^{h+3}}{\sqrt{5}} - 1$$

Despejando h resulta ser logaritmica, entonces para un AVL con n nodos, la altura esta acotada por $O(\log n)$. De manera que la busqueda y otras operaciones tienen complejidad logaritmica en el peor caso.

Red-Black Tree

Es un arbol balanceado donde cada nodo tiene un color asignado, solo puede ser negro o rojo. Sigue las siguientes reglas:

- La raiz siempre es de color negro
- Los nodos rojos no pueden tener hijos del mismo color
- Los caminos de un nodo negro a cada una de sus hojas tienen la misma cantidad de nodos negros
- Todos los nodos nulos son negros

La compejidad de la insercion, borrado y busqueda es O(log(n)).

Altura negra La altura negra de un red-black tree es el numero de nodos negros en un camino de la raiz a una hoja, todas las hojas se cuentan como nodos negros. Entonces la altura del arbol sera:

$$\text{black height} \geq \frac{h}{2}$$

Similarmente, la **black depth** de un nodo se define como la cantidad de nodos negros desde la raiz a dicho nodo.

Insercion Primero, se inserta como un BST. Luego, se arregla cualquier violacion a las reglas de un red-black tree:

- Si el nodo padre es negro, dejo todo como esta
- Si el nodo padre es rojo, podria estar violando alguna regla

Para arreglar alguna violacion, depende del caso:

1. **Uncle is red**: Recolorear el padre y el tio de color negro, al abuelo de rojo. Luego, subir recursivamente chequeando por violaciones.
2. **Uncle is black**: Presenta dos subcasos:
   - **Es hijo derecho**: Rotar a la izquierda con pivot en el padre
   - **Es hijo izquierdo**: Rotar a la derecho con pivot en el abuelo y recolorear

Busqueda Idem a BST

Eliminar Analogamente a la insercion, primero se borra de igual manera a un BST y luego se arregla cualquier violacion de las reglas. Cuando un nodo negro es eliminado, pordria ocurrir que aparesca un **double black**, hay dos casos:

1. **Hermano es rojo**: Rota al padre y recolorea al padre y al hermano.
2. **Hermano es negro**: Se presentan dos subcasos
   - **Los hijos del hermano son negros**: Recolorear al hermano y propagar el double black para arriba.
   - **Por lo menos un hijo del hermano es rojo**: Si el hermano mas lejano es rojo se realiza una rotacion con pivot en el padria y se recolorea. Si el hermano mas lejano es rojo se rota con pivot en el hermano y se recolorea.

   Nota: Las rotaciones son las mismas que para el AVL.

Algunas definiciones

Arbol multicamino M-ario Los nodos guardan hasta M-1 claves de información, con un máximo de M hijos. Cada clave C_i de un cierto nodo será tal que las claves almacenadas en su subárbol izquierdo serán menores y las almacenadas es su subárbol derecho serán mayores que él.

Arboles muticaminos balanceados El equilibrio perfecto resulta muy costoso de mantener y es poco práctico.

En 1970, R. Bayer y E. M. Mc Creight postularon un criterio razonable que permite implementar algoritmos relativamente sencillos para búsquedas, inserciones y eliminaciones.
Para mantener éstos árboles multicaminos balanceados se utiliza una estructura subyacente de la familia de los árboles B, que veremos a continuación

Arbol B de orden N

Un árbol B de orden N es un árbol de búsqueda (ordenado) que cumple con los siguientes axiomas:

- Cada nodo contiene a lo sumo 2 * N claves.
- Cada nodo, excepto la raíz, contiene por los menos N claves.
- Cada nodo o es hoja o tiene M+1 descendientes donde M es el número de claves que posee realmente ese nodo
- Todas las hojas están al mismo nivel
- En cuanto al orden: si un nodo tiene c_1 c_2 ...c_m elementos $c_1 < c_2 <$ ... $< c_m$, pero además para cada c_i $(1 \leq i \leq m)$ los elementos del subárbol izquierdo de ci son menores que c_1 y los elementos del subárbol derecho de c_i son mayores que c_i.

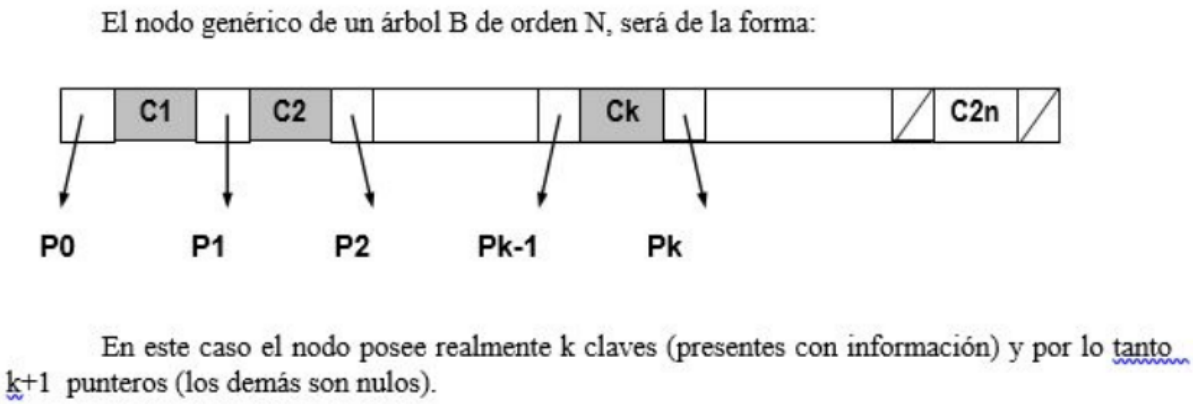

Figure 5: Representacion de un nodo

Algoritmo de busqueda Buscamos la clave X en un nodo. Para ello lo recorremos secuencialmente desde C1 hasta Ck, siendo k el número de claves que realmente posee dicho nodo, hasta que se den alguno de estos casos:

- Si $X < C_1$, como en el nodo las claves están ordenadas no tiene sentido seguir buscando en ese nodo, luego sigo buscando en el subárbol apuntado por P_0
- Si $X = C_i$ para algún $i \leq k$ entonces lo encontré
- Si $C_i < X < C_i + 1$ para algún i prosigo en la búsqueda en el subárbol apuntado por P_i
- Si $C_k < X$ siendo k la cantidad de claves que posee, entonces sigo la búsqueda en el subárbol apuntado por C_k

Si en algún caso el puntero por donde hay que seguir la búsqueda fuera null, entonces el elemento buscado no está.

Algoritmo de insercion Si se quiere insertar una clave X en un árbol B de orden N, se procede de la siguiente manera:

- La inserción siempre se hace en las hojas (para poder detectar si el nodo a insertar ya está presente o no)
- Para insertar se coloca el elemento X en la hoja que corresponda (el nodo debe estar ordenado)
- Si el elemento nuevo hace que la cantidad nueva k sea mayor que el $2 \cdot n$ permitido, el nodo se abre en dos, subiendo la clave del medio al nodo antecesor de dicho nodo. Este algoritmo es recursivo hasta la raíz, o sea si al ubicar la clave del medio en el nodo antecesor ocasiona que el nodo viole la condición de árbol B de orden n ($k > 2 \cdot n$) el procedimiento de repite.

Se mantiene balanceado explotando o contrayendo los nodos, cuando se explota se explota todo hasta la raiz. Nunca se inserta si no es en una hoja. Siempre subo el del medio, notar que es local, solo sube hacia arriba.

Algoritmo de borrado No es tan simple, es complidado pues tambien debe poder contraerse ademas de explotar.

- Si no encuentra en un nodo hoja, se lo reemplaza por unaclave lexicográficamente adyacente, por ejemplo el sucesor in order y se lo elimina de dicha hoja. Si fuera hoja se lo elimina directamente.
- Luego, para la hoja que colaboró el borrado se analiza si cumple las condiciones de árbol B de orden N. Si ha quedado en rojo (tiene menos elementos que los permitidos), se une dicho nodo con su hermano y medio antecesor (el cual es eliminado del nodo al cual pertenece, porque acude en ayuda de su hijo) armando un sólo nodo. Se verifica si cumple las condiciones de árbol B de orden N, y sino se lo particiona subiendo el elemento del medio. Después se analiza qué sucede con el nodo donde estaba su medio antecesor, y se sigue el proceso recurrentemente hasta llegar a la raíz.

  Nota: El borrado no nos lo van a pedir(whatever that means). Cuando dice subir, se refiere a alejarse de la raiz.

Unidad 6: Grafos

La idea es representar relaciones, por ejemplo, el caso del recorrido Euleriano. Actualmente se utilizan para representar rutas, conexiones y trafico. O tambien, con las redes sociales para representar vinculos entre objetos, dada la compejidad de las interrelaciones se crearon alternativas de los algoritmos que ya vimos para que sean mas eficientes.

Ademas, tambien son usados por los instaladores, compiladires y optimizadores.

Tipos

Hay varios tipos de grafos, porque me permiten modelar situaciones distintas. Si se le agregan peso a los ejes tenemos un total de 16 tipos diferentes(8 sin el peso).

No dirigidos

- **Simple**: Entre cada par de nodos hay a lo sumo un eje, no admite lazos.

Dirigido?	Multiplicidad?	Lazos?	Nombre
✗	✗	✗	Simple
✗	✗	✓	*Simple con lazos*
✗	✓	✗	Multi Grafo
✗	✓	✓	Pseudo Grafo
✓	✗	✗	(Simple) Digrafo
✓	✗	✓	*Digrafo con lazos*
✓	✓	✗	Multi Digrafo
✓	✓	✓	Pseudo digrafo

Figure 6: Tabla de los tipos de grafos segun las propiedades

- **Multigrafo**: Entre cada par de nodos puede haber varios ejes, no admite lazos. Normalmente se rotulan los ejes.
- **Pseudografo**: Entre cada par de nodos puede haber varios ejes y admite lazos.

Dirigidos(digrafos)

- **Simple**: Entre cada par de nodos hay a lo sumo un eje, no admite lazos.
- **Multi digrafo**: Entre cada par de nodos puede haber varios ejes, no admite lazos.
- **Pseudo digrafo**: Entre cada par de nodos puede haber varios ejes y admite lazos.

  Nota: Hay algoritmos que dependen de la categoria. No hay clasificacion para simple grafo(o digrafo) con lazo.

Con pesos

Presenta las mismas variantes que los anteriores.

Implementacion

Usando la tabla, vamos a crear una jerarquia de clases para evitar repetir codigo. Si nos enfocamos en un grafo simple, podriamos representarlo como:

- Matriz de adyacencia. Ideal grafos densos.
- Lista de adyacencia. Ideal grafos esparcidos(**sparce**).
- Matriz de incidencia. Idem.
- Lista de incidencia. Idem.

Para relacionar los nodos con un subindice, podriamos usar hashing. Nosotros vamos a usar generalmente

la lista de adyacencia usando factor comun. No a vamos a implementar completamente nosostos, vamos a usar *JGraphT*(usa sparce, por tiempo no implementamos este y los otros tipos, pero no son dificiles), pero tenemos que entender la implementacion.

GraphFactory (sea que lo invoca directamente el usuario o sea que se invoca a partir del GraphBuilder) genera una instancia de SimpleOrDefault y Multi. Ambas clases tienen mucho en común. Sin embargo, difieren en algo:

- `addEdge`: en el caso de SimpleOrDefault, si se vuelve a crear otro eje entre el mismo par de vértices, se ignora. En Multi no se ignora, se crea.
- `removeEdge`: en el caso de SimpleOrDefault, si se indica un par de vértices, con sus propiedades y se lo encuentra, se borra el único eje encontrado. En el caso de Multi se borran todas las apariciones de ese eje con mismas propiedades entre esos vértices.

  Nota: Si no se especifican properties se borra en el caso de SimpleOrDefault el único eje que pudiera existir entre dichos vértices. Si es Multi lanza exception (para evitar ambigüedad). Es responsabilidad del usuario definir equals/hash en la clase que represente las propiedades de los vértices y las propiedades de los ejes.

Testeos de unidad

Como podriamos hacer los testeos de unidad? Por ahi el grafo es el mismo, pero su representacion en memoria es diferente pues internamente usamos un hashing y no podemos saber como quedo realmente.

Recorridos

Los recorridos de nodos se usan para ver los nodos alcanzables a partir de un nodo de interes.

Breadth-First Search (BFS)

Lo vamos a implementar con queue, hay algunos que le agregan un boolean a cada vertice para chequear si fue recorrido, pero es preferible tener una estructura paralella a los vertices booleana. 3

Depth-First Search(DFS)

Similar BFS, pero en vez de queue se usa un stack pues se van visitando desde el proximo, sin chequear si se visitaron todos los del anterior(se agregan al stack).

Dijkstra

Hay varias maneras de implementarlo, pero hay una estructra que java que permite implementarlo con mayor facilidad.

Solo va a funcionar con grafos `WEIGHTED`, idem al que vimos en matematica discreta, se crea una tabla con la distancia de cada nodo al nodo inicial. Con cada ciclo de agregan los adyacentes a la estructura previamente descripta y asi hasta que no quedan mas nodos por visitar.

En la complejidad algoritmica, se puede hacer mas eficiente con un arbol AVL o un RedBlackTree, pero hay una que ya viene implmentada en Java que se conoce como **BinaryQueue** que usa **Binary Heap**.

Un binary heap es un BT completo tal que cada nodo es menor o igual que todos los elementos de sus subarboles(no es un BST). De manera que el minimo esta en la raiz, entonces buscar el minimo es orden 1! Pero al quitarlo hay que buscar alguien que lo reemplaze, Java lo implementa con un arreglo.

Esto hace que sea muy eficiente, ya que si se coloca por niveles en el arreglo, cada elemento del arreglo cumple una propiedad matematica que permite encontrar su antecesor con un simple calculo:

$$\text{Indice antecesor} = \lfloor \frac{i-1}{2} \rfloor$$

Si al insertar, queda mal, solo hay que suapear con el antecesor hasta que quede bien. Peor caso seria O(log(n)), asi que es muy eficiente.